PROCEDIMIENTO DE IMPLEMENTACION

One Language

Proyecto formativo ADSO-3145555 | Version de trabajo: 1.0 | Fecha: 4 de noviembre de 2026

Proposito

Este procedimiento describe como preparar, desplegar y verificar One Language. La solucion separa el cliente web, la aplicacion movil, el backend transaccional, el servicio de reconocimiento de señas y PostgreSQL. El sistema ya tiene un despliegue productivo conocido en https://onelanguage.tech; esta guia distingue esa evidencia de las pruebas locales y no afirma que cada version local este desplegada.

1. Alcance y arquitectura desplegada

Web: React y Vite, servido por Nginx. El navegador usa HTTPS y envia solicitudes REST al backend por el proxy del dominio.

Mobile: React Native con Expo. La app consume el backend por HTTPS y la IA mediante WebSocket seguro.

Backend: Java 17 y Spring Boot; autentica usuarios, aplica reglas de negocio, gestiona permisos, accesibilidad e historial.

IA: servicio FastAPI en Python. Recibe secuencias de landmarks por WebSocket y responde predicciones. El endpoint de estado es /health.

Base de datos: PostgreSQL 15 con cambios de esquema administrados por Liquibase. Los datos persisten en volumen Docker.

Produccion: Nginx del VPS termina TLS y enruta trafico. La web se publica internamente en 127.0.0.1:8080, el backend en 127.0.0.1:8084 y la IA en 127.0.0.1:8000. PostgreSQL no se publica a Internet. Backend, IA, base de datos y web se administran con proyectos/archivos Compose separados y una red Docker privada; no se debe suponer un unico Compose global.

2. Requisitos previos

Para desarrollo: Git, Docker Desktop con WSL2 funcional, Node.js/npm para web y Expo, JDK 17 y Maven para backend, Python compatible con los requisitos versionados del servicio IA, y acceso a los repositorios.

Para produccion: acceso SSH autorizado al VPS, DNS y certificados TLS vigentes, Docker Engine/Compose, Nginx, red Docker privada, volumen de datos y artefactos del modelo IA (lsc_sequence_model.keras, lsc_labels.json y lsc_config.json).

Mantenga los secretos fuera del repositorio. Configure variables de entorno requeridas por cada servicio a partir de su README y archivos de ejemplo. Nunca copie contraseñas, JWT_SECRET, tokens ni datos personales en este documento, logs o comandos compartidos.

3. Preparacion y ejecucion local

1. Actualice cada repositorio desde la rama de integracion acordada por el equipo y revise git status antes de cambiar archivos.

2. Revise los README y archivos Compose de frontend, backend, IA y base de datos. Cree los archivos .env locales desde los ejemplos y complete valores de desarrollo, sin reutilizar secretos productivos.

3. Inicie PostgreSQL y espere que el healthcheck indique estado healthy. Confirme que backend y base de datos comparten una red Docker o que la URL JDBC apunta al host correcto.

4. Inicie el backend con Java 17/Maven o el Compose documentado; confirme el puerto 8084 y que el log indique conexion correcta a PostgreSQL y aplicacion de Liquibase.

5. Inicie la IA con el comando indicado en su README; confirme GET /health en el puerto 8000, el estado del modelo y sus etiquetas. Verifique que la variable de origen permitido incluya el origen local HTTPS usado por Vite cuando corresponda.

6. Inicie la web con npm ci y npm run dev dentro de web. Use el origen configurado en Vite y compruebe que las rutas /api y /auth llegan al backend y que el WebSocket seguro llega a la IA.

7. Para movil, instale dependencias en mobile y ejecute Expo segun el perfil local. Configure las URL publicas o de red local para backend e IA; localhost dentro del telefono no representa el PC.

8. Pruebe registro, inicio de sesion, consulta de usuario, inicio/cierre de traduccion y acceso al historial. Autorice camara y microfono solo cuando la funcion lo solicite.

4. Despliegue web en el VPS

1. Coordine ventana de mantenimiento y confirme acceso al VPS, DNS, TLS, imagen actual y estado de servicios.

2. Guarde la referencia de la imagen/contenedor actual y valide que el volumen de PostgreSQL y el volumen de archivos del backend existen. No elimine volumentes ni bases de datos para actualizar la web.

3. En /opt/onelanguage/frontend, actualice el checkout a la version aprobada de main sin descartar cambios locales. Si hay cambios del servidor, respaldelos y aclaremoslos antes de continuar.

4. Construya la imagen desde el contexto web con el Dockerfile de produccion y los valores de build revisados. No incruste secretos en argumentos de build.

5. Recree solo el contenedor web, conectado a onelanguage_net y publicado como 127.0.0.1:8080:80. Mantenga backend, IA y PostgreSQL fuera de este cambio.

6. Valide `docker ps`, `curl -I http://127.0.0.1:8080`, `curl -I https://onelanguage.tech` y `curl -I https://onelanguage.tech/downloads/OneLanguage.apk`. Revise que el APK servido sea el artefacto aprobado.

7. Ejecute pruebas funcionales: cargar landing, login, navegar a traduccion, confirmar llamada API y WebSocket, revisar historial y cerrar sesion. Examine errores de consola y logs de Nginx/backend/IA.

8. Si falla una comprobacion, restaure la imagen anterior y el archivo de configuracion Nginx respaldado; confirme nuevamente HTTP, TLS, API y acceso a datos. No revierta migraciones de base de datos automaticamente.

5. Verificacion y evidencia

La URL productiva conocida es https://onelanguage.tech. El servicio IA ofrece https://ai.onelanguage.tech/health y el WebSocket de reconocimiento usa wss://ai.onelanguage.tech/ws/recognize. El endpoint de salud valida disponibilidad basica, no la calidad de reconocimiento.

En la validacion local del 4 de noviembre de 2026, el backend ejecuto 41 pruebas con 0 fallos y 0 errores; seis pruebas estaban excluidas por defecto (una E2E y cinco de concurrencia). Al habilitar el perfil de integracion contra PostgreSQL temporal, la E2E y las cinco pruebas de concurrencia pasaron. La prueba HTTP verifico registro (201), login (200), /api/users/me autenticado (200) y no autenticado (401), ademas de almacenamiento de contraseña con BCrypt. Liquibase valido y aplico 21 changesets en la base temporal. Estos resultados son evidencia local, no una certificacion de produccion.

6. Operacion y responsabilidades

El responsable de version debe aprobar el commit y registrar la ventana. El operador valida respaldos, despliegue y rollback. El responsable de base de datos conserva volumen, verifica migraciones y respalda/restaura con procedimiento aprobado. El responsable funcional prueba login, traduccion, permisos e historial. Las credenciales de correo y rotacion de secretos corresponden al administrador autorizado de esas cuentas.

7. Criterios de cierre

Se considera implementada una version cuando la imagen corresponde a la version aprobada, contenedores relevantes estan saludables, TLS y rutas REST/WebSocket responden, una cuenta de prueba puede iniciar sesion y las operaciones criticas quedan verificadas. La evidencia debe registrar fecha, version/commit, responsable, resultado y rollback; nunca incluir secretos o datos reales.